

プログラミングI 演習

Computer Programming I: Exercises

第2回 変数と入出力

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

本日の講義

2

本日の目標

- 変数と変数の型を覚える
- (数学とは異なる) 代入文を覚える
- 変数を利用した演算ができるようになる
- scanf文を使って変数への入力, printf文を使って変数の内容を出力することができる

本日の資料

1. 前回のおさらい
2. 変数と代入文
3. 変数の型
4. 変数に対する入出力
5. 演習課題

1. 前回のおさらい: プログラムの実行

3

ソースコード
=ソースプログラム

ASCII文字(半角の英数字や記号等)によって記述

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!");
    return 0;
}
```

ソースコードを
作成・支援するソフトウェア=エディタが必要

次ページ



実行可能ファイル
=機械語プログラム

実行可能なデータは
CPU毎に決められた機械語によって記述

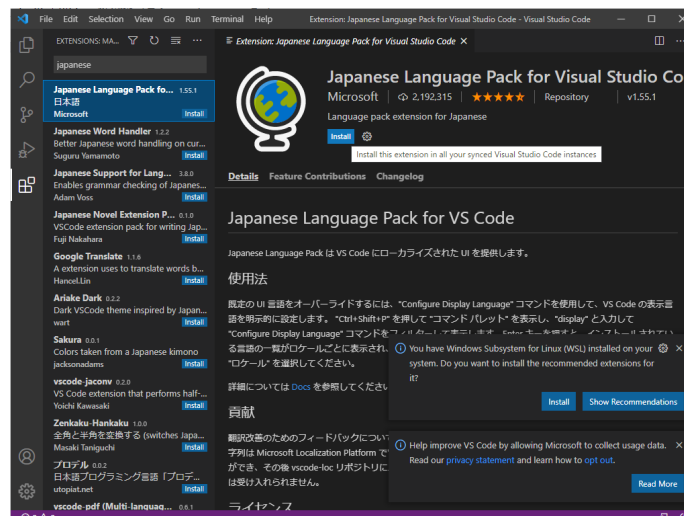
```
4D 5A 90 00 03 00 00 00-04 00 00 00 FF FF 00 00
58 00 00 00 00 00 00 00-40 00 00 00 00 00 00
00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
00 00 00 00 00 00 00 00-00 00 00 ED 00 00 00
0E 1F BA 0E 00 B4 09 CD-21 68 01 4C CD 21 54 68
69 73 20 70 72 6F 67 72-61 6D 20 63 61 6E 6E 6F
74 20 62 65 20 72 75 6E-20 69 6E 20 44 4F 53 20
6D 6F 64 65 2E 0D 0D 0A-24 00 00 00 00 00 00
10 39 E2 36 54 58 8C 65-54 58 8C 65 54 58 8C 65
54 58 8D 65 5A 5A 8C 65-73 9E F1 65 57 58 8C 65
73 9E F6 65 55 58 8C 65-73 9E F1 65 52 58 8C 65
73 9E E1 65 55 58 8C 65-73 9E E2 65 FF 58 8C 65
73 9E F0 65 55 58 8C 65-73 9E F4 65 55 58 8C 65
52 69 63 68 54 58 8C 65-00 00 00 00 00 00 00
50 45 00 00 4C 01 04 00-6D A7 91 47 00 00 00 00
00 00 00 00 E0 00 02 21-08 01 08 00 00 EE 0C 00
00 CE 00 00 00 00 00 00-92 45 04 00 00 10 00 00
00 60 0C 00 00 00 DF 77-00 10 00 00 00 02 00 00
06 00 00 00 06 00 00 00-00 00 00 00 00 00 00 00
00 B0 00 00 00 04 00 00-61 6C 0E 00 03 00 04 01
00 00 04 00 00 10 00 00-00 00 10 00 00 10 00 00
00 00 00 00 10 00 00 00-9C 00 0C 00 57 94 00 00
```

ソースコードを機械語に
変換するソフトウェアが必要

1. 前回のおさらい: プログラムの実行

4

エディタ=ソースコードの入力・編集ソフト
Visual Studio Codeの使用方法を紹介しました

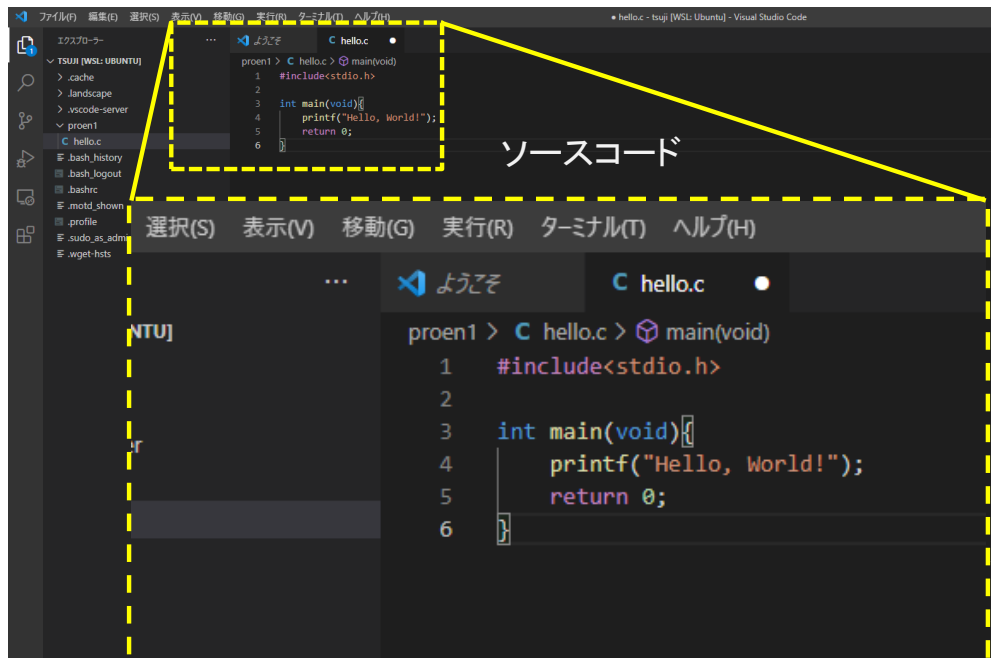


他にもエディタはありますので
自分で判断できる人は好きなものを使用してください

1. 前回のおさらい：プログラムの実行

5

・プログラムを書こう



1. 前回のおさらい：プログラムの実行

6

・プログラムを書こう

エディタで開いたら・・・

下記プログラムを書いてみる

```
#include<stdio.h>

int main(void)
{
    printf("Hello, World!");
    return 0;
}
```

資料からのコピー＆ペーストでは、かなりの確率でエラーになります
打ち直しましょう

1. 前回のおさらい: プログラムの実行

7

補足: 世界一有名なプログラム「Hello, world!」

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!");
    return 0;
}
```

「stdio.h」とは、標準的入出力(=Standard Input/Output, 例えばprintf)を行うために必要な関数が定義・格納されているファイル

#include <stdio.h>は「stdio.h」を読み込むという宣言

「main」関数

プログラムが処理を開始するときに最初に呼ばれる関数

「printf」関数

文字を出力する関数
この演習の場合はコンソール上に文字が出力される

1. 前回のおさらい: プログラムの実行

8

ソースコード
= ソースプログラム

ASCII文字(半角の英数字や記号等)によって記述

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!");
    return 0;
}
```

自前PCやLinuxサーバで、各自好きなエディタを使用してコーディング。
サクラエディタ, Eclipse, Visual studio, sublime text, emacs, vimなど

実行可能ファイル
= 機械語プログラム

実行可能なデータは
CPU毎に決められた機械語によって記述

```
4D 5A 90 00 03 00 00 00-04 00 00 00 FF FF 00 00
B8 00 00 00 00 00 00 00-40 00 00 00 00 00 00
00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
00 00 00 00 00 00 00 00-00 00 00 00 E0 00 00
0E 1F BA 0E 00 64 09 CD-21 B8 01 4C CD 21 54 68
69 73 20 70 72 6F 67 72-61 6D 20 63 61 6E 6E 6F
74 20 62 65 20 72 75 6E-20 69 6E 20 44 4F 53 20
6D 6F 64 65 2E 00 0D 0A-24 00 00 00 00 00 00
10 39 E2 36 54 58 8C 65-54 58 8C 65 54 58 8C 65
54 58 8D 65 5A 5A 8C 65-73 9E F7 65 57 58 8C 65
73 9E F6 65 55 58 8C 65-73 9E F1 65 52 58 8C 65
73 9E E1 65 95 58 8C 65-73 9E E2 65 FF 58 8C 65
73 9E F0 65 55 58 8C 65-73 9E F4 65 55 58 8C 65
52 69 63 68 54 58 8C 65-00 00 00 00 00 00 00
50 45 00 00 4C 01 04 00-6D A7 91 47 00 00 00
00 00 00 00 E0 00 02 21-08 01 08 00 00 BE 0C 00
00 CE 00 00 00 00 00 92-45 04 00 00 10 00 00
00 60 0C 00 00 00 DF 77-00 10 00 00 02 00 00
06 00 00 00 06 00 00 06-00 00 00 00 00 00 00
00 B0 00 00 00 04 00 00-61 6C 0E 00 03 00 40 01
00 00 04 00 00 10 00 00-00 10 00 00 10 00 00
00 00 00 00 10 00 00 00-9C 00 0C 00 57 94 00 00
```

ソースコードを機械語に
変換するソフトウェア=コンパイラが必要

次ページ

1. 前回のおさらい: プログラムの実行

9

• コンパイル (compile) と実行

gcc ./hello.c → 作成したプログラムを機械語に翻訳
./a.out → プログラム実行

1. 前回のおさらい: プログラムの実行

10

まとめ

■プログラミング言語

人間がプログラムを作りやすくするための言語
「C」(Programming Language “C”、C言語)を使う

■コンパイラ

GCC (GNU Compiler Collection)

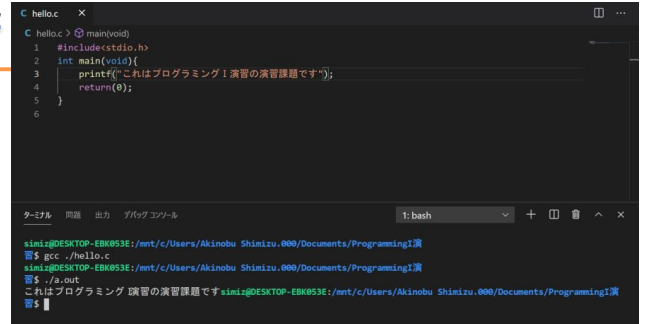
■コンパイル実行環境

- WSL(Ubuntu 20.04 LTS) ※WSLはwindowsの場合. macOSでは仮想環境は使っていない(ただし、Dockerで環境構築を行ったひとを除く)
- Visual Studio Code(...上のターミナルからgcc)

演習1-1 「最初のプログラム」の改造

```
#include<stdio.h>

int main(void)
{
    printf("Hello, World!");
    return 0;
}
```



" "内にある文字を好きなものに
変更し、出力させてみよう

注意

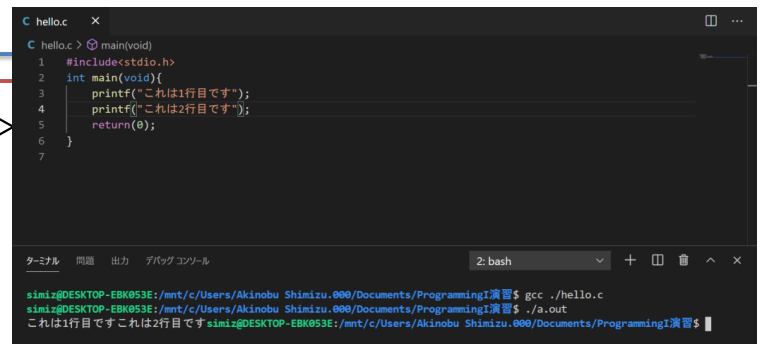
- 演習のスライドのコードをそのままコピーしても動作しない場合があります
- PDFのスライド中にプログラムでは禁止されている文字(例えば全角の空白など)が紛れているためです。
- 必ず自分で半角英数字を使ったコードを入力しましょう(※タイピングの練習も重要です)

演習課題

演習1-2 2行以上の出力

```
#include<stdio.h>

int main(void)
{
    printf("これは1行目です");
    printf("これは2行目です");
    return 0;
}
```



printf文を2つ書き、出力してみよう。
そして、出力が2行にならないことを確認しよう

※ 補足: " "の間なら全角日本語は入力可能

演習1-3 改行文字の利用

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
printf("これは1行目です¥n");
```

```
printf("これは2行目です");
```

```
return 0;
```

```
}
```

「これは1行目です」の後に「¥n」をいれて実行してみよう
「¥n」を改行文字と呼びます

補足:「¥n」は「\n(バックスラッシュn)」のことですが、
日本語環境では¥が表示されます
Macではoptionキーと¥を同時押しで入力

2. 変数と代入文

変数の宣言

```
int x;
```

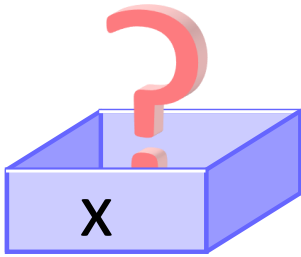
```
#include<stdio.h>
int main(void){
    int x;

    x=7;
    x=x+2;
    return 0;
}
```

- 宣言は関数の中のならべく早い段階で行う
- コンパイラに
「int型」の変数「x」を作ってくださいとお願いする
xは整数(-2,147,483,648~2,147,483,647)
の値を取りますよ、という宣言
- メモリ上の部屋に「x」という名前がつき、データ
を出し入れすることができる

※intやdoubleなど型によって部屋の広さなどが変化

変数の宣言直後



```
#include<stdio.h>
int main(void) {
    int x;
    x=7;
    x=x+2;
    return 0;
}
```

- メモリ上の部屋「x」ができる
- 宣言した時点では内容は不定



変数を宣言したら、初期化をする習慣をつけましょう

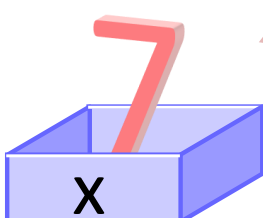
2. 変数と代入文

変数の初期化

x = 7;

- 「=」を代入演算子と呼ぶ
- 変数「x」に値7を代入
- 変数への最初の代入を初期化という

```
#include<stdio.h>
int main(void) {
    int x;
    x=7;
    x=x+2;
    return 0;
}
```



左辺 = 右辺



補足: 宣言と同時に初期化する方法もある

```
int main(void)
{
    int x=7;

    x=x+2;
    return 0;
}
```

左辺の変数に右辺の値(計算結果)を代入する

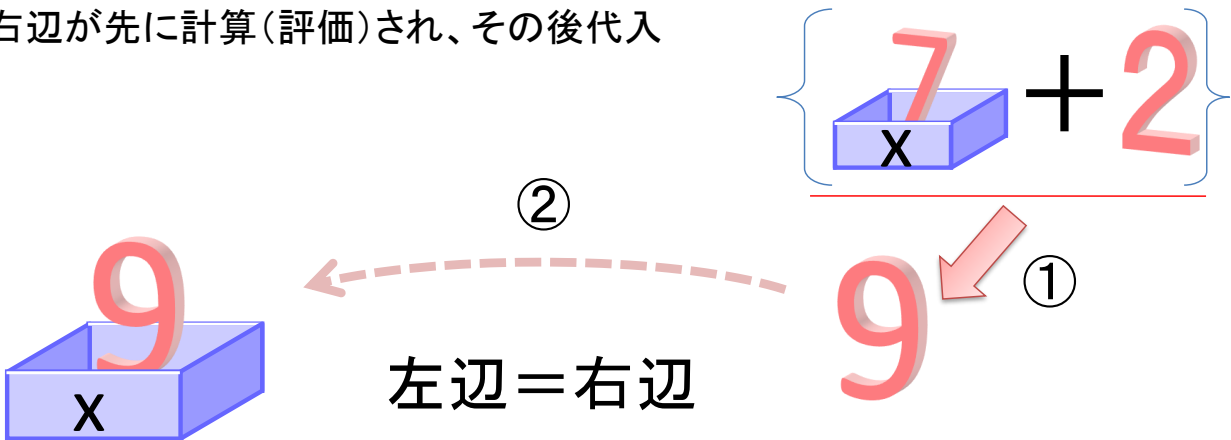
代入演算子の性質

$x = x + 2;$

- 「=」は等号ではない！
(両辺が等しくないのは明らか)
- 右辺が先に計算(評価)され、その後代入

```
#include<stdio.h>
int main(void) {
    int x;

    x=7;
    x=x+2;
}
```



2. 変数と代入文

演算子: 数値同士の計算を行わせる命令

四則演算

演算子	意味: 使用例
+	加算: $1 + 2$, $a + b$
-	減算: $1 - 2$, $a - b$
*	乗算: $1 * 2$, $a * b$
/	除算: $1 / 2$, a / b

特殊な演算

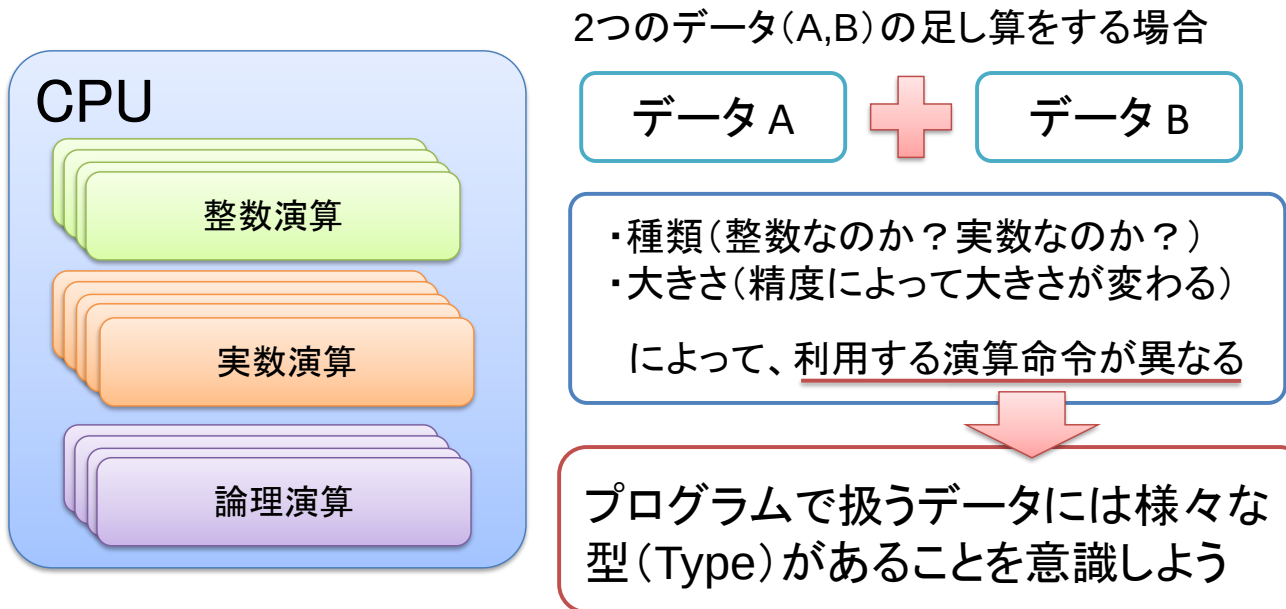
演算子	意味: 使用例
=	代入: $a = 1$, $a = b = 10$
+= (-=, *=, /= もある)	加算代入: $a += 10$ ($a = a + 10$ と同じ)
++ (-- もある)	インクリメント: $a++$ (a を1つ増やす)
%	剰余: $a \% b$ (a を b で割った時のあまり)

3. 変数の型 (Type)

19

型 (Type) とは

CPU(中央演算装置)には様々な種類の数値に対応した
計算を行う回路が用意されている



3. 変数の型 (Type)

20

C言語でよく使われる型 (1)

int型 (integer・整数型)

- ・整数を表す型
- ・通常のWindowsでの大きさは32bit (=4Byte)
(表現範囲: -2147483648 ~ +2147483647)

float型 (単精度浮動小数点型)

- ・実数を表す型
- ・通常のWindowsでの大きさは32bit (=4Byte)
(符号付. 絶対値の表現範囲: $1.17549 \times 10^{-38} \sim 3.40282 \times 10^{38}$. 精度: 約6桁)

double型 (倍精度浮動小数点型)

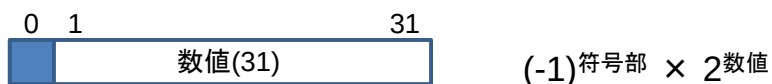
- ・実数を表す型
- ・通常のWindowsでの大きさは64bit (=8Byte)
(符号付. 絶対値の表現範囲: $2.22507385850720 \times 10^{-308} \sim 1.79769313486231 \times 10^{308}$. 精度: 約15桁)
- ・floatのサイズを倍にして、精度を高めたもの

3. 変数の型 (Type)

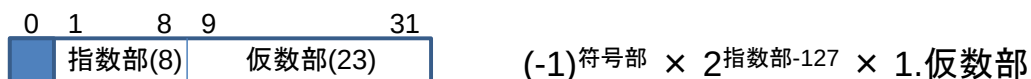
21

C言語でよく使われる型(1)の内部表現  符号(0 or 1(負数))

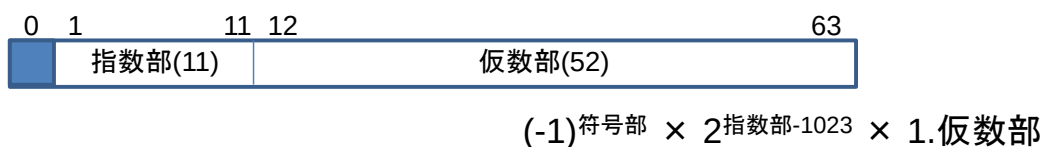
int型 (integer・整数型)



float型 (単精度浮動小数点型)



double型 (倍精度浮動小数点型)



浮動小数点数内部表現シミュレーターの例
<https://tools.m-bsys.com/calculators/ieee754.php>

3. 変数の型 (Type)

22

C言語でよく使われる型(2)

char型 (character、文字型)

- ・半角英数字(1文字)を表す型
- ・大きさは8bit (=1Byte)
- ・第10回の「文字列」の演習で詳しく説明

void型 (空型)

- ・「何もしません」ということを表す型
- ・関数(第6回の演習で説明)でも利用される型
- ・main関数の後ろに(void)と書いてあるが、

「()の中には何もしません。書き忘れてでもありません。」

ということを、コンパイラに教えている

```
int main(void){ }
```

出力関数

printf ("出力書式") 関数

ディスプレイに文字を出力する関数の1つ

- 出力したい文字列を" "で囲み、関数に渡す
- 「¥n」は改行を表す制御文字

```
#include<stdio.h>
int main(void){
    printf("Hello, world!¥n");
    return 0;
}
```



実行画面

Hello, world!

4. 変数に対する入出力

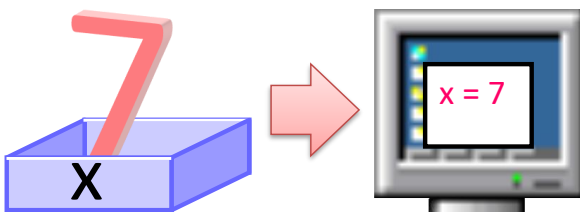
変数の出力

```
#include<stdio.h>
int main(void){
    int x = 7;
    printf("x=%d¥n", x);
    return 0;
}
```

printf ("x = %d¥n", x);

変数「x」の値を整数の10進表記で表示

- " "に囲まれた中にある「%d」の位置に変数の内容が表示される
- 複数の変数(式)を表示可能
例: printf("%d + %d = %d ¥n", x, y, x+y);



変数の型とprintfの書式

※表示書式は変数の型によって変える必要がある

変数の型	printfの書式
int	%d (8進数:%o,16進数:%x)
float	%f
double	%lf
char	%c

```
#include<stdio.h>
int main(void){
    int x = 65;
    printf("%d,%f,%c¥n", x,x,x );
    return 0;
}
```

1つの変数を異なる書式で出力してみよう。それぞれの形式に合わせて出力されるはず。変数の型と書式が一致していない場合には何が起こるかを(エラーコードも含めて)見てみよう。

出力:

4. 変数に対する入出力

小数型の整形

小数を出力するときに、桁数の指定をする

```
#include<stdio.h>
int main(void){
    float f = 1.23456789;
    printf("%f,%.2f,%.10f¥n", f,f,f );
    return 0;
}
```

小数点以下の「何桁」まで値を表示するかを指定することができる

出力:

注: 表示桁数を増やしたからと言って、変数の精度が上がるわけではない。
変数のbit長は変わっていないことに注意

変数の入力

```
#include<stdio.h>
int main(void){
    int x;
    scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

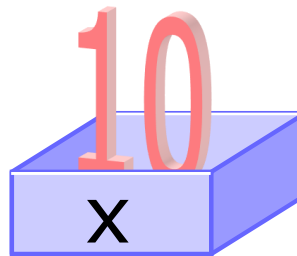
scanf("%d", &x);

キーボードで入力した整数値を x に代入
※書式を変数の型と一致させる必要あり

- 「printf」関数とよく似ているが、**変数の前に「&」をつけるのを忘れない**
- 「&」は、後ろの変数「x」のメモリ上の位置（部屋番号、アドレス）を教えるためにつける（※第7回で詳しく説明）



「1」「0」「↵」と入力



4. 変数に対する入出力

scanfの書式と型

入力時の書式の指定はprintfのときと同様

型	書式
int	%d (8進数:%o, 16進数:%x)
float	%f
double	%lf
char	%c

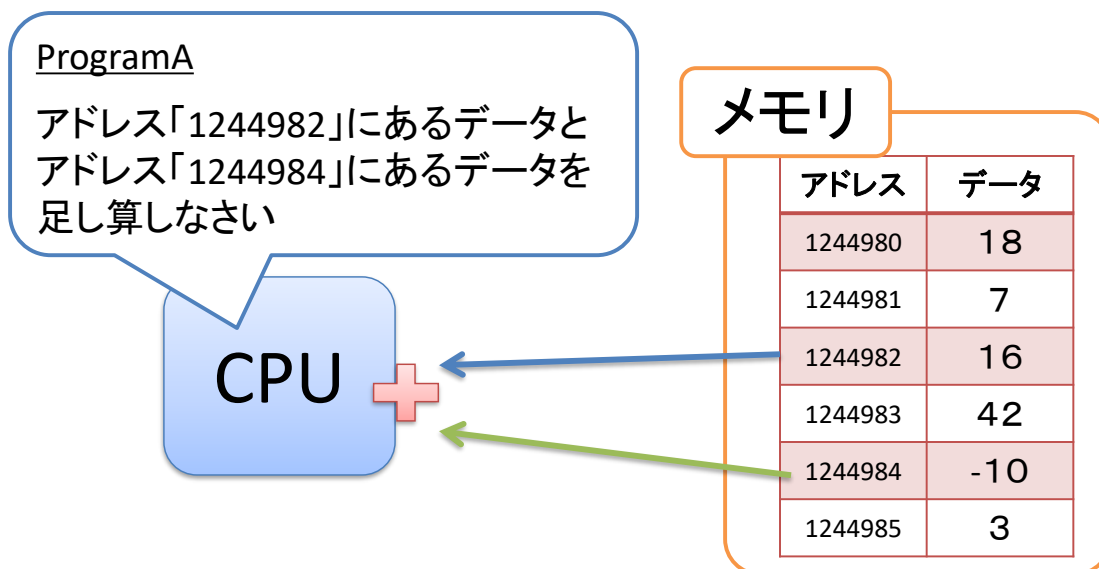
入力では%lfとしないとダメ

```
#include<stdio.h>
int main(void){
    int x;
    scanf("%d", &x);
    scanf("%lf", &x);
    return 0;
}
```

異なるサイズの型に値を入力する
→ 変数からデータがあふれる
→ 他のデータを壊す危険がある

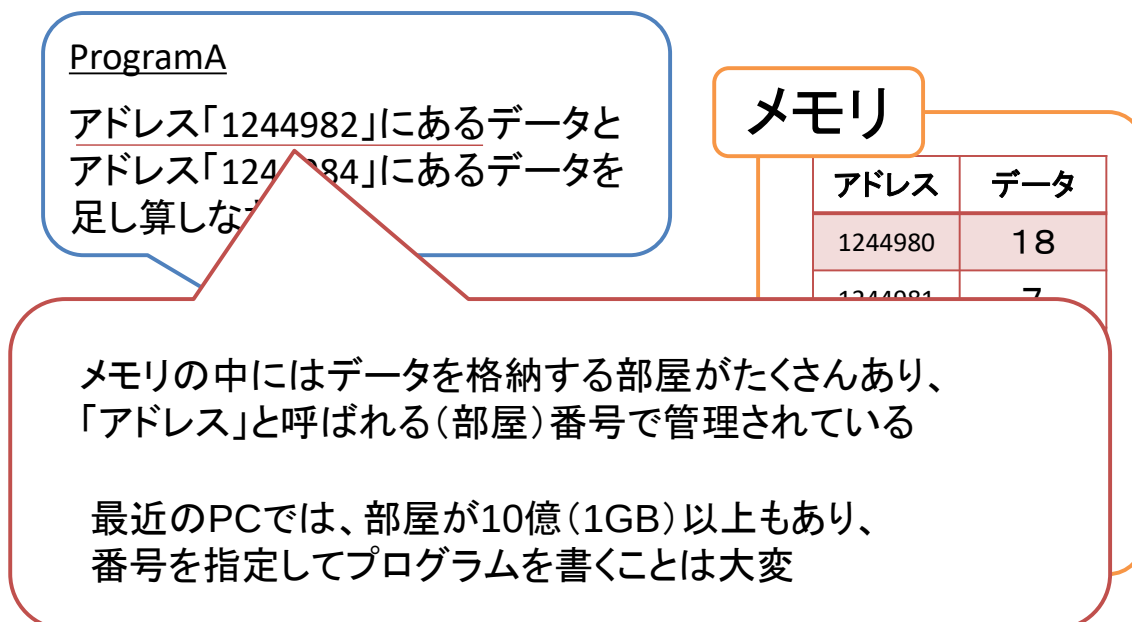
変数 (variable) とメモリの関係

計算機で扱うデータは必ずメモリに保存されていて
プログラムから呼び出し、計算する



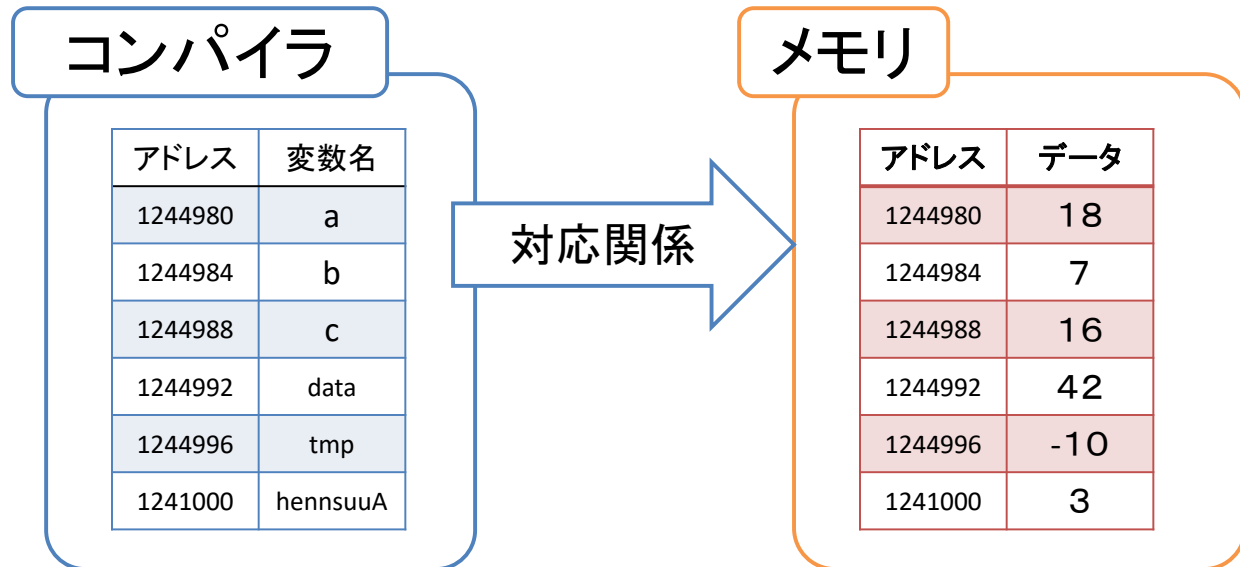
変数 (variable) とメモリの関係

計算機で扱うデータは必ずメモリに保存されていて
プログラムから呼び出し、計算する



変数 (variable) とメモリの関係

メモリの「部屋」に分かりやすい「名前」を付ける仕組み



ここから演習課題です

演習2-1 「変数内の値」の出力

```
#include<stdio.h>

int main(void)
{
    int a;
    a = 2;
    printf("aの値は%dです", a);
    return 0;
}
```

この通りプログラムを書いて、
%dのところに、aの値が出力される
ことを確認しよう。

5. 演習課題

演習2-2 「変数の型」による出力の違い

```
#include<stdio.h>

int main(void)
{
    int a;
    a = 2;
    printf("aの値は%dです", a);
    return 0;
}
```

変数の型、値、書式を変えて出力してみよう。
int型、float型、double型
の3つを試みましょう。

演習2-3 変数を利用した演算

```
#include<stdio.h>

int main(void)
{
    int a, b;
    a = 2;
    b = 4;
    a = a + b;
    printf("aの値は%dです", a);
    return 0;
}
```

変数を2つ以上宣言し、変数を利用した演算を行ってみましょう。
型、値、演算子、書式をいろいろと変えてみてください。

演習2-4 変数への入力

```
#include<stdio.h>

int main(void)
{
    int a, b;
    scanf("%d", &a);
    scanf("%d", &b);
    a = a + b;
    printf("aの値は%dです", a);
    return 0;
}
```

変数を2つ以上宣言し、宣言した変数に値を入力して、演算を行ってみましょう

5. 演習課題 (※発展課題の提出は必須ではありません) 39

発展2-1 入力された実数を小数第1位で四捨五入する

実行例

小数を入力:3.72
四捨五入すると4です。

基本方針:

- ・ double型の変数へ小数を格納する
- ・ int型の変数を別に用意する
- ・ 「 int型 = (int)double型 」
という代入をすることで、小数点以下が無視される
- ・ 入力された実数の小数点以下を無視するのは「切り捨て」
どうすれば「四捨五入」になる？ →(ヒント)無視する前に何かする

型変換(=キャスト)という
操作を行っています。

5. 演習課題

40

発展2-1 入力された実数を小数第1位で四捨五入する

※以下は、異なる型同士の計算や代入をする場合の例です。この結果をよく見て、計算、代入、キャストの過程で何が起きているかを理解し、プログラムを作りましょう。

```
#include<stdio.h>
int main(void)
{
    int      IntVal = 3;
    float     FloatVal = 1.2345;

    IntVal = (int)FloatVal;
    FloatVal = (float)IntVal;

    printf("%d,%f¥n", IntVal, FloatVal);
}
```

型変換
(変換先の型)
を、変換したい変数・数値の
前につける

型変換をしないと...

例えば、「float f = 3/9;」は「float = int / int」なので
「3/9」の計算結果(0.333...)もint型に変換されてしまい
「f = 0.00000」という代入が行われてしまう

本日の説明内容はおしまいです。
課題に取り組んでください。